Let's say our `all_cars` index was created with two shards, and because the data is growing exponentially, the index is now overloaded. To mitigate the risk of slow queries and degrading performance, we want to create a new index with more space. To do this, we can split the index into a new one with more room and additional primary shards.

Before we invoke the split operation on the `all_cars` index, we must make sure the index is disabled for indexing purposes—that is, we need to change the index to read-only. The following code helps by invoking the `_settings` API.

Listing 6.41 Making sure the index is read-only

```
PUT all_cars/_settings                          <-- Uses the _settings API
{
  "settings":{
    "index.blocks.write":"true"                 <-- Closes the index for writing operations
  }
}
```

Now that the prerequisite of making the index non-operational is complete, we can split it by invoking the `_split` API. This API expects the source and target indexes:

```
POST <source_index>/_split/<target_index>
```

Now, let's split the index into a new index (`all_cars_new`).

Listing 6.42 Splitting the `all_cars` index

```
POST all_cars/_split/all_cars_new               <-- _split expects a target index.
{
  "settings": {
    "index.number_of_shards": 12                <-- Sets the number of shards on the new index
  }
}
```

This request kicks off the splitting process. The splitting operation is synchronous, meaning the client's request waits for a response until the process is completed. Once the split completes, the new index (`all_cars_new`) will have all the data as well as additional space because more shards were added to it.

As mentioned earlier, the splitting operation comes with certain rules and conditions. Let's look at some of those:

- *The target index must not exist before this operation.* In addition to the configuration we provide in the request object while splitting (listing 6.42), an exact copy of the source index is transferred to the target index.
- *The number of shards in the target index must be a multiple of the number of shards in the source index.* If the source index has three primary shards, the target index can be defined with a number of shards that is a multiple of three (three, six, nine, etc.).